

Eightfold Engineering Intern (Jul-Dec 2026) — Assignment

Multi-Source Candidate Data Transformer — Technical Design

1. Pipeline Breakdown

extract → **normalize** → **merge** → **confidence** → **project** → **validate**

- **extract**: each source (Recruiter CSV, ATS JSON, GitHub profile) has its own extractor returning a uniform {source, fields} record.
- **normalize**: per-field functions — phone → E.164 (phonenumbers, with real-number validation), date → YYYY-MM (dateutil), skill → lowercased canonical name via a synonym map (e.g. js/reactjs → javascript/react).
- **merge**: candidates are bucketed by normalized email, falling back to phone, then name; conflicting field values are resolved via a source-priority table.
- **confidence**: per-field and overall scores computed from source reliability plus corroboration across sources.
- **project**: a config-driven layer reshapes the canonical record into the requested output schema (rename/filter/omit), fully decoupled from the merge engine.
- **validate**: final JSON is checked against the canonical schema plus targeted rules (email/phone format, duplicate detection, confidence bounds) before being returned.

2. Canonical Schema & Normalized Formats

- Fixed fields: candidate_id, full_name, emails[], phones[], headline, current_company, title, skills[{name, confidence, sources[]}], provenance[], overall_confidence.
- Phones: E.164 format via phonenumbers; numbers that fail real-number validation (e.g. placeholder area codes) are dropped rather than stored as a wrong-but-confident value.
- Dates: normalized to YYYY-MM.
- Skills: lowercased and mapped through a synonym table so equivalent terms (JS/JavaScript, ReactJS/React) collapse into one canonical skill.
- Emails: lowercased and trimmed before matching, so case differences across sources (Jane.Doe@ vs jane.doe@) still merge correctly.

3. Merge & Conflict-Resolution Policy

- Match key: normalized email first; phone second; normalized name as last resort. This lets the same person be recognized across structured and unstructured sources even with formatting differences.
- Conflict resolution: a per-field source-priority table (below) decides the winning value deterministically — no averaging or random tie-breaking.
- Every chosen value carries provenance: {field, source, method, confidence}, so any output value is traceable back to its origin source and the rule that selected it.
- The same inputs always produce the same canonical output, since field selection relies on fixed priority rules and deterministic normalization — no randomness or order-dependence.

Field	Priority order	Reasoning
email / phone	ATS > Recruiter CSV > Resume	ATS data is typically HR-verified
full_name / title	ATS > Recruiter CSV > GitHub	Same verification reasoning
skills	GitHub (repository languages) > Resume (self-reported skills)	Repository languages provide evidence of technologies used, while resume skills are self-reported

4. Confidence Scoring

confidence = base_source_reliability × (1 + 0.15 × num_corroborating_sources), capped at 1.0

- Base reliability is assigned per source (ATS highest, GitHub and Recruiter CSV mid, Resume lowest) reflecting how independently verifiable that source's data tends to be.
- A field's score increases slightly when multiple sources agree on the same value, rewarding corroboration without letting volume alone inflate trust.

5. Runtime Custom-Output Config

- A JSON config (fields[], include_confidence, on_missing) is read by the projection layer at runtime — no code changes needed to reshape output.
- Each field entry supports path (output name), from (source path in the canonical record), type (with auto string/array coercion), and required.
- on_missing supports null / omit / error, applied uniformly during projection, after the canonical record is already fully built.

6. Edge Cases Handled (and Descoped)

- Missing/absent source file → extractor returns an empty result; pipeline continues without crashing.
- Malformed JSON / unparseable CSV row → caught and skipped; treated as empty contribution, never invented data.
- Same candidate, case-mismatched email across sources → still merges into one profile via lowercased email matching.
- Field present in one source but missing in another (e.g. phone missing in CSV, present in ATS) → filled from the available source.
- Invalid/placeholder phone numbers (failing real-number validation) → dropped to null, never stored as confident-but-wrong.
- Descoped: Resume PDF/DOCX parsing and LinkedIn extraction (no public API) — same extractor interface, drop-in addition later.